SHIPFLEET – DIGITAL MARITIME FLEET OPERATIONS AND PERFORMANCE MONITORING

A PROJECT REPORT

Submitted by

AMIRTHA P

(Registration No: 310624243018)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIG ENCE AND DATA SCIENCE



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY,

CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"SHIPFLEET – DIGITAL MARITIME FLEET OPERATIONS AND PERFORMANCE MONITORING"** is the Bonafide work of **"AMIRTHA P (310624243018)",** who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. T. Kalai Selvi
Easwari Engineering College,
Ramapuram, Chennai – 600089.

SUPERVISOR
Ms. D. S. K. Harshitha
Easwari Engineering College,
Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

Certificate Of Viva-Voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled **"SHIPFLEET – DIGITAL MARITIME FLEET OPERATIONS AND PERFORMANCE MONITORING"** submitted by **Ms. AMIRTHA P (Reg. No. 310624243018)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology was conducted on **23-10-2026** in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Date:

ABSTRACT

NavisFleet (FleetOps Maritime) is a full-stack web platform that lets dispatchers track commercial vessels, schedule voyages, and manage fuel, cargo and maintenance in one secured portal. Maritime shipping carries over 80% of global merchandise trade, yet many operators still rely on siloed spreadsheets, periodically refreshed AIS maps, unauthenticated internal tools and weather data kept apart from vessel tracks.

The system is built with React, TypeScript, Node.js, Express, Socket.io and MongoDB. It simulates real-time AIS telemetry by dead reckoning, with speed multipliers of 1x, 2x, 5x and 10x and pause/resume control. Positions are streamed to an interactive Leaflet map on an Esri dark maritime basemap, which carries marine weather overlays: tropical cyclone cones, Significant Wave Height zones, Beaufort wind vectors and precipitation radar.

Role-Based Access Control (RBAC) separates Administrators from General Users. Port Directories, Crew Certification, Audit Logs and global administration are restricted to administrators using JSON Web Token (JWT) authorization with server-side role checks. Passwords are hashed with bcrypt, and every create, update and delete action is written to an audit log.

By combining WebSocket streaming, persistent storage, dead-reckoning calculation and reactive GIS mapping, NavisFleet delivers an end-to-end digital dispatch solution with near-real-time fleet synchronisation.

Keywords: AIS telemetry, fleet management, RBAC, JWT, Socket.io, Leaflet, marine weather overlays, MERN stack

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr. T. KalaiSelvi**, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

We are extremely thankful to our Project Guide **Ms. D. S. K. Harshitha**, Department of Artificial Intelligence and Data Science for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	iv
	ACKNOWLEDGEMENT	v
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
1.	**INTRODUCTION**	1
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	2
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	3
	1.5 MOTIVATION	5
2.	**LITERATURE SURVEY**	6
	2.1 LITERATURE REVIEW	6
	2.2 EXISTING SYSTEM	7
	2.3 PROPOSED SYSTEM	8
3.	**SYSTEM ANALYSIS**	9
	3.1 SYSTEM REQUIREMENTS	9
	3.1.1 Functional Requirements	9
	3.1.2 Non-Functional Requirements	10
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	11
4.	**METHODOLOGY**	14
	4.1 SYSTEM ARCHITECTURE	14

	4.2 MODULE DESCRIPTION	15
	4.3 CLASS DIAGRAM	18
	4.4 SEQUENCE DIAGRAM	19
5.	**RESULTS AND DISCUSSION**	20
	5.1 IMPLEMENTATION OVERVIEW	20
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	20
6.	**CONCLUSION & FUTURE WORK**	29
	6.1 CONCLUSION	29
	6.2 FUTURE ENHANCEMENTS	30
	APPENDIX: SOURCE CODE	31
	REFERENCES	57

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 3.1	Non-Functional Requirements Matrix	10
Table 3.2	Software Requirements Specification	11
Table 3.3	Hardware Requirements Specification	12
Table 4.1	Functional Modules and Core Responsibilities	15
Table 4.2	Database Collections and Schema Field Model	16

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 4.1	Three-Tier System Architecture of the NavisFleet Platform	14
Figure 4.2	Class Diagram of User, Vessel, Voyage, and FuelRecord Entities	18
Figure 4.3	Sequence Diagram for JWT Authentication and Real-Time WebSocket Telemetry	19
Figure 5.1	Public Maritime Landing Page and Fleet Operations Gateway	36
Figure 5.2	Role-Based Authentication & Modal Gate	36
Figure 5.3	Directorate Master Dashboard with Real-Time KPI Cards & Telemetry	37
Figure 5.4	Fleet Strength & Vessel Inventory Management Console	37
Figure 5.5	International Voyage Scheduling and Cargo Dispatch Interface	38
Figure 5.6	Interactive Maritime GIS Map with Live Weather & Telemetry Overlays	38
Figure 5.7	Cargo Manifest Management & Bill of Lading Roster	39
Figure 5.8	Bunker Fuel Consumption, Efficiency Metrics & Cost Analytics	39
Figure 5.9	Preventative & Corrective Fleet Maintenance Work Orders	40
Figure 5.10	Marine Crew Roster & STCW Certification Compliance Directory	40

CHAPTER 1

INTRODUCTION

This chapter deals with Fleet, a real-time maritime fleet logistics and AIS telemetry platform. It explains the problem addressed, the objectives, the scope of the work and the motivation behind it.

1.1 OVERVIEW

Maritime shipping carries over 80% of global merchandise trade by volume and is the backbone of international commerce. Efficient dispatch of container ships, dry bulk carriers, LNG tankers and RoRo vessels depends on accurate vessel positions, engine metrics, fuel consumption and cargo manifests across high-seas routes.

NavisFleet is an end-to-end maritime management platform that centralises and automates fleet logistics. It provides live geographic tracking through Automatic Identification System (AIS) simulation, interactive meteorological overlays, voyage routing, cargo manifest tracking, fuel bunkering efficiency analysis and preventative maintenance logging. The frontend follows Single-Page Application (SPA) principles; the backend is asynchronous and event-driven.

1.2 PROBLEM DEFINITION

Traditional maritime logistics platforms suffer from five technical and operational problems:

- **Disjointed systems:** voyage scheduling, fuel bunkering, cargo documents and maintenance orders sit in separate spreadsheets or legacy on-premise software.
- **High tracking latency:** tools that query periodic batch data show stale ship positions and inaccurate Estimated Time of Arrival (ETA) projections.
- **No weather awareness:** operators lack real-time overlays of cyclones, high swell zones and gale winds next to vessel tracks.
- **Poor access separation:** without strict Role-Based Access Control (RBAC), non-administrative operators can view or alter crew contracts, port directories and audit trails.
- **Basemap licensing issues:** many map layers add watermarks that obscure navigation coordinates.

1.3 OBJECTIVE

The objectives of NavisFleet are:

1. Build a real-time fleet dashboard showing vessel coordinates, speed, heading and status using Leaflet and WebSocket streams.
2. Provide an adjustable AIS simulation engine that can pause, resume and accelerate dead-reckoning movement (1x, 2x, 5x, 10x).
3. Add marine meteorological layers: Significant Wave Height (Hs), Beaufort wind vectors, precipitation radar and cyclone warning cones with advisories.
4. Enforce RBAC so that Port Directories, Crew Management and System Audits are reserved for administrators.
5. Offer full CRUD workflows for Voyages, Cargo Manifests, Fuel Operations and Maintenance Work Orders, with audit logging and CSV reporting.

1.4 SCOPE OF THE PROJECT

- **Vessel tracking:** live visualisation of commercial vessels on major routes (Suez Canal, Malacca Strait, Panama Canal, English Channel, Cape of Good Hope).
- **Meteorological intelligence:** wave height contours, wind barbs and active cyclone cones on the tracking map.
- **Logistics operations:** voyage dispatching, cargo allocation, bunker fuel analysis and maintenance tracking.
- **Security and authorisation:** JWT authorisation, bcrypt password hashing, input validation and session handling.
- **Data export:** CSV reports for fleet inventory, voyage logs, fuel expenditure and maintenance schedules.

The simulation uses synthetic vessel data; integration with a live AIS feed is outside the present scope.

1.5 MOTIVATION

In ocean shipping, a delayed alert caused by a severe storm or engine wear can lead to heavy financial loss and environmental harm. Fuel is also one of the largest operating costs of a voyage, so small gains in routing and bunkering efficiency matter. A unified, open-standard and highly reactive web platform gives dispatchers and fleet superintendents situational awareness, lower bunkering expense and shorter transit times, and restricts sensitive data to authorised staff.

CHAPTER 2

RELATED WORK

This chapter deals with the research and existing systems related to maritime fleet tracking. It reviews AIS-based tracking, GIS mapping and WebSocket streaming, and examines the limitations of current maritime portals. It ends by presenting the proposed NavisFleet system and how it addresses those limitations.

2.1 LITERATURE REVIEW

Research on Maritime Intelligent Transportation Systems (M-ITS) stresses combining spaceborne AIS data with geographic information systems. The International Maritime Organization (IMO) notes that digitised voyage dispatch and continuous fuel monitoring help reduce a vessel's Carbon Intensity Indicator (CII) and greenhouse gas emissions. Studies of web architectures report that WebSocket event streaming outperforms HTTP polling in multi-client GIS platforms by lowering server overhead and payload size. Such streaming suits live vessel tracking, where positions change every few seconds.

2.2 EXISTING SYSTEM

Existing Maritime Portals (Legacy Vhf Radio Dispatch, Standalone Erp Modules And Static Ais Trackers) Have Four Limitations:

- **Periodic Batch Refresh:** Maps Update Every 5 To 15 Minutes, So Course Changes Appear Late.
- **Uncoupled Modules:** Fuel Records, Crew Rosters And Bills Of Lading Are Stored Apart From The Vessel Map.
- **Static Map Layers:** Storm Tracks And Sea-State Swells Are Absent From The Main Navigation Screen.
- **Rigid Permissions:** Every Logged-In Operator Has The Same Access, Which Creates Data Governance Risk.

2.3 PROPOSED SYSTEM

NavisFleet addresses these limitations with five features:

- **Event-driven architecture:** Socket.io pushes telemetry updates to browser maps every simulation tick (2.5 seconds), with no page reload.
- **Simulation control:** users pause, resume and accelerate dead-reckoning movement at 1x, 2x, 5x or 10x.
- **Integrated weather overlays:** cyclones, significant wave heights, wind vectors and precipitation radar are drawn over an Esri dark maritime basemap.
- **Granular RBAC:** standard users work with their own voyages, cargo and fuel logs; Port Directories, Crew Rosters, Audit Trails and System Resets are for administrators only.
- **Audit trail:** create, update and delete actions are recorded with timestamp, user and IP metadata.

CHAPTER 3

SYSTEM ANALYSIS

This chapter deals with the requirements of the NavisFleet system. It lists the functional requirements, such as authentication, live fleet display, weather overlays, voyage management and data export, and the non-functional requirements of performance, security and availability. It also specifies the software stack and hardware needed to build and run the system.

3.1 SYSTEM REQUIREMENTS

Functional Requirements

- **Authentication and sessions:** secure signup, login with JWT issuance, bcrypt password verification and session persistence.
- **Live fleet display:** map rendering of active vessels with heading-rotated icons, status badges and waypoint polylines.
- **Simulation control:** play/pause and speed multipliers (1x, 2x, 5x, 10x), synchronised between client and server over sockets.
- **Weather overlays:** selectable Wind Flow, Wave Height, Precipitation Radar and Cyclone Warning Cone layers with advisory popups.
- **Voyage and dispatch management:** create, view, update and delete voyages with cargo manifests, origin and destination ports and ETAs.
- **Fuel and maintenance tracking:** record bunkering, compute fuel efficiency in nautical miles per metric ton, log repair costs and track work-order status.
- **Role-based navigation:** Port Directory and Crew Management limited to administrators; administrative status indicators hidden from standard users.
- **Data export:** CSV files for Fleet, Voyages, Fuel and Maintenance.

Non-functional requirements

Category	Requirement
Performance	Map rendering under 100 ms; telemetry message delivery under 50 ms (design targets)
Security	bcrypt hashing (salt factor 10); Bearer-token protection on API endpoints; role validation on administrative operations
Availability	In-memory fallback store when the external MongoDB is unreachable
Compatibility	Responsive layout for Chrome, Edge, Firefox, Safari and tablet viewports

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Software requirements

Component	Specification
Operating system	Ubuntu 20.04+, Windows 10/11 or macOS Monterey+
Frontend	React 19.x with TypeScript
Styling and icons	Tailwind CSS 4.x, Lucide React
Mapping	Leaflet 1.9.x, Esri ArcGIS tiles, RainViewer radar API
Backend runtime	Node.js v18.x or v20.x
Server framework	Express 4.x with tsx runtime
Real-time engine	Socket.io 4.x

Component	Specification
Database	MongoDB with Mongoose ODM (in-memory fallback engine)
Tooling	Vite 6.x, TypeScript compiler, ESLint

Hardware requirements

Component	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher (quad-core recommended)
RAM	4 GB minimum, 8 GB recommended
Storage	500 MB free disk space
Network	Broadband, minimum 2 Mbps for map tile streaming

CHAPTER 4

SYSTEM DESIGN

This chapter deals with the design approach of NavisFleet. It presents the three-tier architecture, the functional modules and the database schema. It also uses a class diagram and a sequence diagram to show how the main components interact during login and live telemetry streaming.

4.1 SYSTEM ARCHITECTURE

NavisFleet follows a decoupled three-tier client-server structure. The client tier holds the React pages, the Leaflet map and the Socket.io client. The application tier runs Express and Socket.io on Node.js with JWT and RBAC middleware, controllers, and the AIS dead-reckoning service. The data tier stores users, vessels, voyages, cargo, fuel records, maintenance orders, ports, crew, alerts and audit logs in MongoDB.

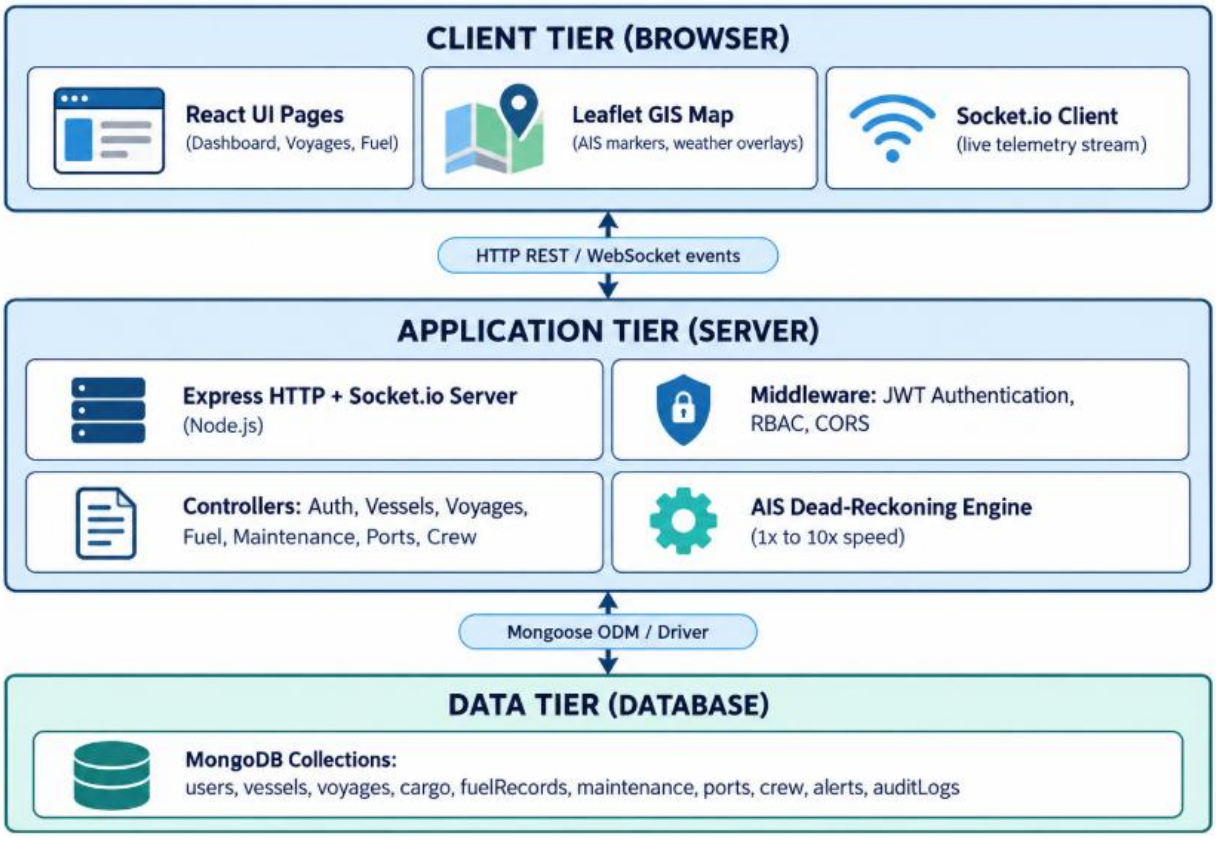


Figure 4.1: the client tier talks to the application tier over HTTP REST and WebSocket events, and the application tier reaches MongoDB through the Mongoose ODM.

4.2 MODULE DESCRIPTION

Module	Responsibility
Authentication and Identity	Registration, login, JWT issuance, bcrypt hashing and RBAC claims (admin or user)
AIS Telemetry and Simulation	Computes dead-reckoning position deltas from heading and speed, scales each step by the speed multiplier (0x, 1x, 2x, 5x, 10x) and broadcasts coordinates on the vessel:telemetry socket event
GIS Map and Meteorological	Leaflet container, heading markers, route polylines, Esri dark tiles and weather overlays (wind vectors, wave contours, cyclone warnings)
Voyage and Logistics Dispatch	Schedules voyages between ports, assigns cargo manifests and calculates transit progress
Bunker Fuel Operations	Tracks bunkering logs, fuel type (VLSFO, MGO, LNG), cost per metric ton and fuel efficiency in nautical miles per metric ton
Maintenance and Fleet Health	Scheduled and emergency work orders, priorities, costs, technicians and vessel status updates
Administrative and Audit Governance	Port Directories, Crew Certifications, system data resets and audit logs, restricted to administrators

4.3 DATABASE SCHEMA / DATA MODELS

Collection	Fields
User	_id, name, email, password (hashed), role (admin or user), department, createdAt
Vessel	_id, vesselId, name, imoNumber, type, flag, builtYear, deadweightTonnage, currentLocation (lat, lng, heading, speedKnots), status, fuelCapacity, currentFuel
Voyage	_id, voyageId, vesselId, originPort, destinationPort, departureDate, estimatedArrival, status, distanceNauticalMiles, distanceCovered, assignedCargoIds, createdBy
Cargo	_id, cargoId, description, weightTons, hazardClass, shipper, consignee, status
FuelRecord	_id, vesselId, fuelType, quantityMT, unitPriceUSD, totalCostUSD, fuelEfficiencyNMPerMT, port, date, createdBy
Maintenance	_id, vesselId, type, severity, description, startDate, status, costUSD, technician, createdBy
Port	_id, portId, name, country, city, portCode, lat, lng, capacityTEU, status
Crew	_id, name, role, vesselId, nationality, certification, contractExpiry, status

Collection	Fields
AuditLog	_id, action, resource, resourceId, details, userId, userEmail, ipAddress, timestamp

4.4 CLASS DIAGRAM

The class diagram shows the four core classes. A User manages many Vessels and dispatches many Voyages; each Vessel executes many FuelRecords.

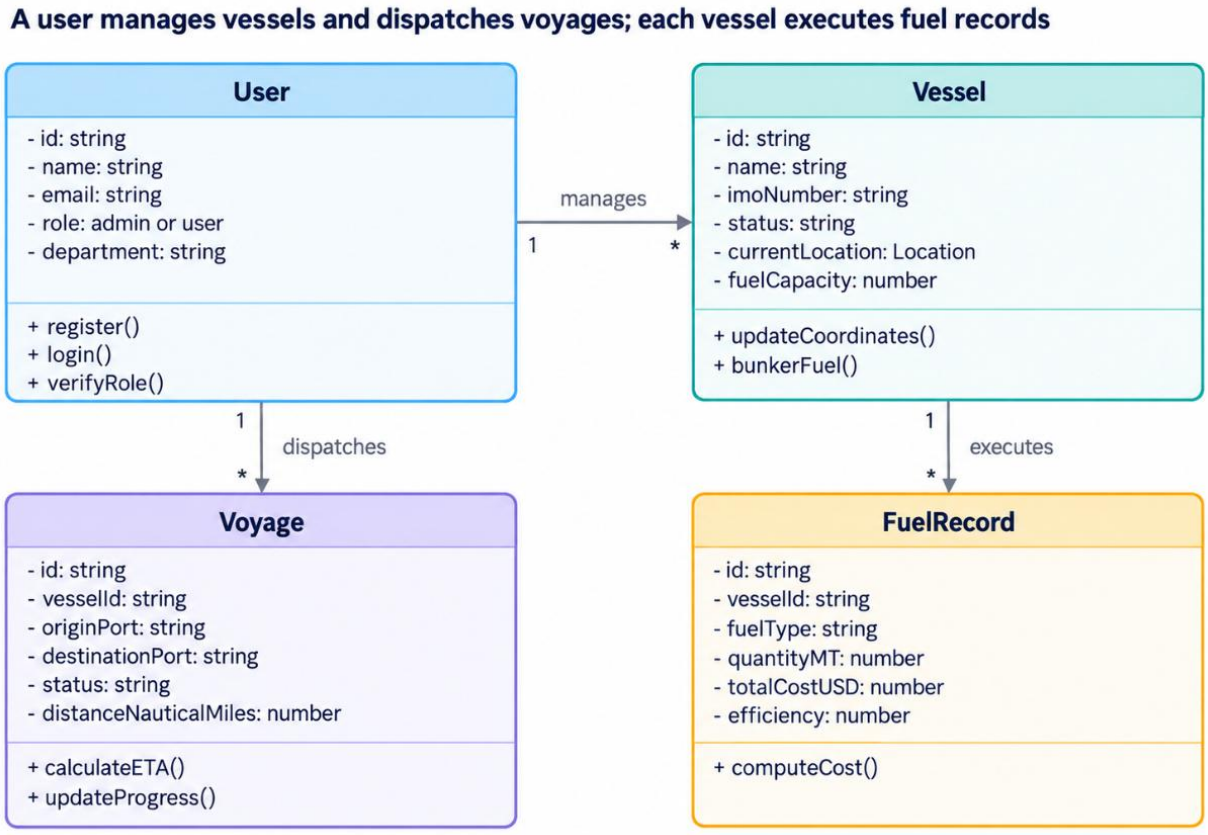


Figure 4.4: class diagram of User, Vessel, Voyage and FuelRecord with their one-to-many relationships.

4.5 SEQUENCE DIAGRAM

The sequence diagram covers login and the telemetry stream: the browser authenticates through the Express API, connects a WebSocket with its token, and then receives a vessel:telemetry event every 2.5 seconds from the simulation loop

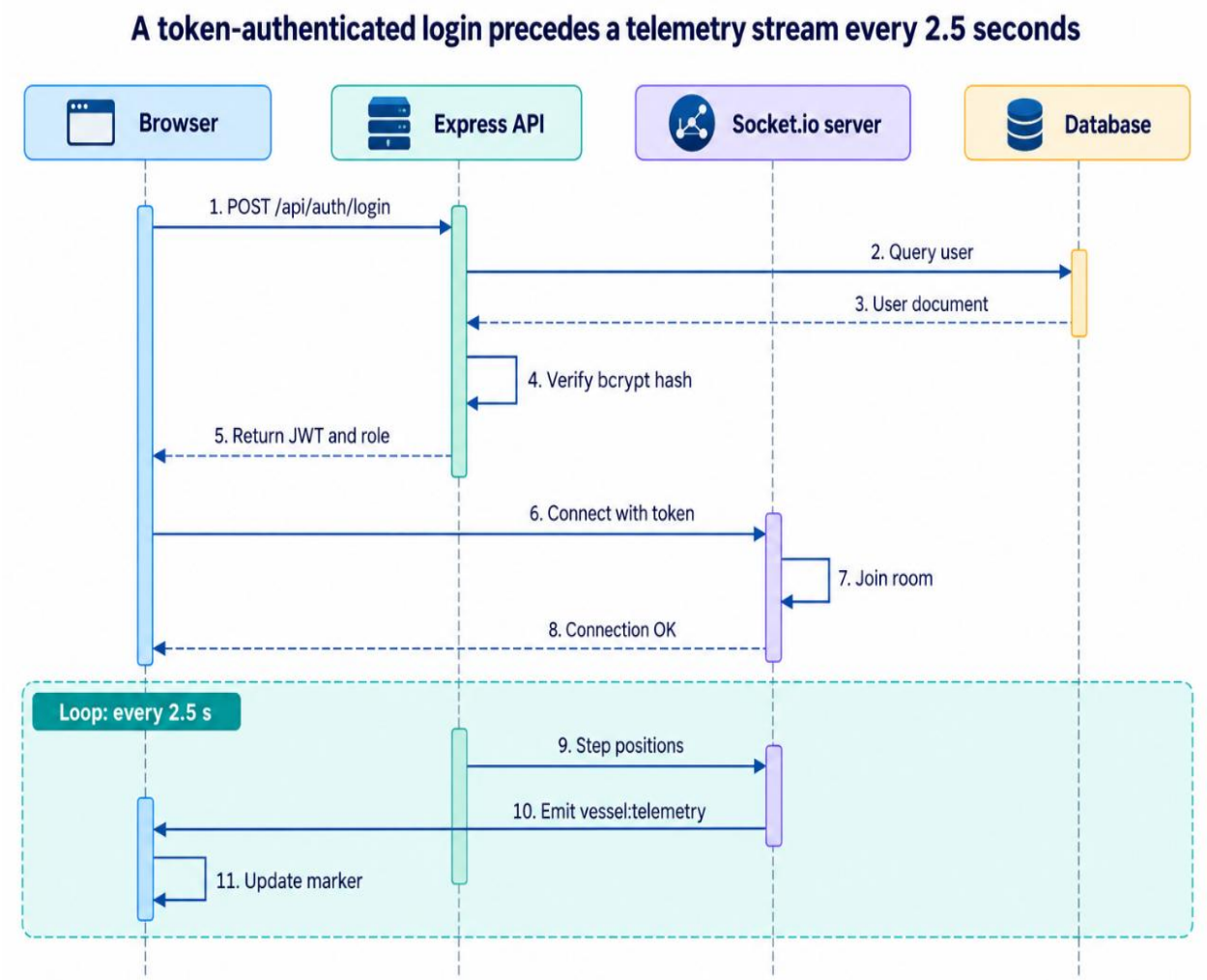


Figure 4.5: after login the browser opens a WebSocket with its token and receives vessel:telemetry events from the simulation loop.

CHAPTER 5

IMPLEMENTATION

This chapter deals with the implementation of the design and the results it delivers. It covers the AIS simulation engine with 1x to 10x speed control, the Esri dark basemap, the weather overlays and role-based access isolation, and explains the key functions through code. It also discusses the limits of the work, such as the use of synthetic data.

5.1 IMPLEMENTATION OVERVIEW

The implementation combines an Express TypeScript server with a Vite-powered React single-page application. Four milestones define it:

- **Dynamic AIS engine:** runs in server.ts on an interval timer. At 1x, vessels advance at normal dead-reckoning speed; at 2x, 5x or 10x the position step is scaled; a speed of 0 halts calculation.
- **Esri dark maritime basemap:** LeafletMap.tsx uses the Esri World Dark Gray Base and Reference layers in place of Carto tiles, which removes watermarked tiles.
- **Tactical weather overlays:** src/data/marineWeather.ts provides structured data for active cyclones (Category 1 to 4), wave height zones, Beaufort wind vectors and RainViewer radar reflectivity.
- **RBAC isolation:** the Port Directory and Crew Management links appear only under the isAdmin condition in Sidebar.tsx, and the routes are protected in App.tsx and routes.ts. The top-bar status pills (online users, MongoDB status) are also administrator-only.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

1. AIS telemetry simulation step (server.ts)

Every 2.5 seconds the server selects vessels whose status is In Transit, converts the heading to radians and computes a position offset from speed in knots multiplied by the speed multiplier. The latitude delta uses the cosine of the heading and the longitude delta uses the sine. The new position is saved and broadcast to all clients as a vessel:telemetry event. The constant 0.04 is a

scaling factor chosen so that the simulated motion is visible on screen; it is not a physical conversion.

```
setInterval(() => {
 if (simulationSpeed <= 0) return; // Paused state
 try {
  const inTransitVessels = db.find('vessels', (v) => v.status === 'In Transit');
  if (inTransitVessels.length === 0) return;

  for (const vsl of inTransitVessels) {
   const headingRad = (vsl.currentLocation.heading * Math.PI) / 180;
   const speedOffset = (vsl.currentLocation.speedKnots / 3600) * 0.04 * simulationSpeed;
   const latDelta = Math.cos(headingRad) * speedOffset;
   const lngDelta = Math.sin(headingRad) * speedOffset;

   const updatedLocation = {
    ...vsl.currentLocation,
    lat: Number((vsl.currentLocation.lat + latDelta).toFixed(4)),
    lng: Number((vsl.currentLocation.lng + lngDelta).toFixed(4)),
    updatedAt: new Date().toISOString()
   };

   db.findByIdAndUpdate('vessels', vsl._id, { currentLocation: updatedLocation });

   io.emit('vessel:telemetry', {
    vesselId: vsl._id,
    vesselName: vsl.name,
    location: updatedLocation,
    simulationSpeed
   });
  }
 } catch (err) {
  console.error('Telemetry simulation error:', err);
 }
}, 2500);
```

2. JWT verification and RBAC middleware (src/server/routes.ts)

Authenticate reads the Bearer token, verifies it with the server secret and attaches the decoded user (id, email, role) to the request; a missing or invalid token returns 401. requireRole then checks that the user's role is in the allowed list and returns 403 otherwise.

```
export const authenticate = (req: Request, res: Response, next: NextFunction): void => {
  const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) {
    res.status(401).json({ success: false, message: 'Authentication required' });
    return;
  }
  try {
    const decoded = jwt.verify(token, JWT_SECRET) as { id: string; role: string; email: string };
    (req as any).user = decoded;
    next();
  } catch {
    res.status(401).json({ success: false, message: 'Invalid or expired session token' });
  }
};

export const requireRole = (roles: string[]) => {
  return (req: Request, res: Response, next: NextFunction): void => {
    const userRole = (req as any).user?.role;
    if (!userRole || !roles.includes(userRole)) {
      res.status(403).json({ success: false, message: 'Forbidden: Insufficient privileges' });
      return;
    }
    next();
  };
};
```

3. Login controller (hardened)

The login route looks up the user by lower-cased email and compares the password with bcrypt only. Plaintext comparison and any environment-key override are not allowed, because they would let anyone holding that key sign in as any account. Both unknown email and wrong password return the same 401 message so that accounts cannot be enumerated.

```
router.post('/auth/login', async (req: Request, res: Response): Promise<void> => {
  const { email, password } = req.body;
  if (!email || !password) {
    res.status(400).json({ success: false, message: 'Email and password required' });
    return;
  }

  const user = db.find('users', (u) => u.email.toLowerCase() === email.toLowerCase().trim())[0];
  const isMatch = user ? await bcrypt.compare(password, user.password) : false;
  if (!isMatch) {
    res.status(401).json({ success: false, message: 'Invalid credentials' });
    return;
  }

  const token = jwt.sign(
    { id: user._id, email: user.email, role: user.role, name: user.name },
    JWT_SECRET,
    { expiresIn: '7d' }
  );
  res.json({ success: true, data: { user: { _id: user._id, name: user.name, email: user.email, role: user.role }, token }
});
});
```

4. Voyage update with ownership check (hardened)

A standard user may update only voyages they created, an administrator may update any voyage, and only whitelisted fields are accepted. This stops a caller from changing createdBy or other protected fields through the request body.

```
const UPDATABLE = ['originPort', 'destinationPort', 'departureDate', 'estimatedArrival', 'status', 'distanceCovered'];

router.put('/voyages/:id', authenticate, (req: Request, res: Response): void => {
  const user = (req as any).user;
  const voyage = db.findById('voyages', req.params.id);
  if (!voyage) {
    res.status(404).json({ success: false, message: 'Voyage not found' });
    return;
  }
  if (user.role !== 'admin' && voyage.createdBy !== user.id) {
    res.status(403).json({ success: false, message: 'Forbidden: not your voyage' });
    return;
  }

  const changes = Object.fromEntries(
    Object.entries(req.body).filter(([key]) => UPDATABLE.includes(key))
  );
  const updated = db.findByIdAndUpdate('voyages', req.params.id, changes);
  res.json({ success: true, data: updated });
});
```

CHAPTER 6

CONCLUSION & FUTURE WORK

This chapter deals with a summary of what the project has achieved and how it can be extended. It restates how real-time communication, GIS mapping and role-based access control solve fragmented fleet tracking, and proposes future work such as machine learning ETA prediction, live AIS integration and automated weather routing.

6.1 CONCLUSION

NavisFleet demonstrates a modern, scalable and responsive maritime dispatch system. By uniting real-time WebSocket communication, GIS mapping with meteorological overlays and strict Role-Based Access Control, it addresses the main pain points of fragmented fleet tracking. Dispatchers and administrators get a single view of live vessel positions, simulation control and logistics records, on a basemap free of licensing watermarks.

The present version uses simulated vessel positions and synthetic weather data; the architecture is ready for live AIS and forecast feeds.

6.2 FUTURE ENHANCEMENTS

- **Machine learning ETA prediction:** train LSTM networks on historical AIS waypoints, sea currents and fuel burn rates to predict port arrival windows.
- **Satellite weather imagery:** ingest geostationary infrared imagery (GOES, Himawari) for live cloud cover.
- **Blockchain bill of lading (e-BL):** use smart contracts for tamper-evident transfer of cargo ownership.
- **Automated weather routing:** compute routes around severe sea states (high Hs) to reduce fuel use and hull stress.
- **Live AIS integration:** replace the dead-reckoning simulation with a real AIS data provider.
- **Latitude-corrected motion:** scale longitude steps by the cosine of latitude for more accurate dead reckoning away from the equator.

APPENDIX

SOURCE CODE & CONFIGURATION

1. PROJECT DIRECTORY STRUCTURE

```
shipfleet-operations/
├── server.ts              # Express server, Socket.io and AIS engine
├── package.json
├── vite.config.ts
├── tsconfig.json
├── index.html
└── src/
    ├── main.tsx           # React bootstrap
    ├── App.tsx            # Routing and role navigation
    ├── index.css          # Tailwind imports
    ├── context/
    │   ├── AuthContext.tsx    # Session, login and token state
    │   └── SocketContext.tsx  # Socket.io connection and telemetry
    ├── components/
    │   ├── Navbar.tsx  Sidebar.tsx  LeafletMap.tsx  LoginModal.tsx
    │   ├── LandingPage.tsx  Toast.tsx  ConfirmDialog.tsx
    ├── pages/
    │   ├── DashboardPage.tsx  RouteTrackingPage.tsx  VesselsPage.tsx
    │   ├── VesselDetailsPage.tsx  VoyagesPage.tsx  CargoPage.tsx
    │   ├── FuelPage.tsx  MaintenancePage.tsx  AlertsPage.tsx
    │   ├── ReportsPage.tsx  HistoryPage.tsx
    │   ├── PortsPage.tsx      # Admin only
    │   ├── CrewPage.tsx       # Admin only
    │   └── UsersPage.tsx      # Admin only
    ├── data/marineWeather.ts  # Cyclones, wave height and wind datasets
    ├── services/api.ts        # Axios REST client
    ├── types/client.ts        # TypeScript interfaces
    └── server/
        ├── routes.ts          # REST controllers and RBAC
        └── db.ts              # Database connection and in-memory fallback
```

2. BACKEND PACKAGE INSTALLATION

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcryptjs socket.io
```

3. DATABASE CONNECTION (CONFIG/DB.JS)

```
 const mongoose = require('mongoose');

const connectDB = async () => {
 try {
  await mongoose.connect(process.env.MONGO_URI);
  console.log('MongoDB Connected');
 } catch (err) {
  console.error('Database connection failed:', err.message);
  process.exit(1);
 }
};

module.exports = connectDB;
```

4. ENVIRONMENT VARIABLES (.ENV)

```
PORT=3000
MONGO_URI=mongodb://localhost:27017/maritime_fleet_db
JWT_SECRET=my_super_secret_jwt_key_123
```

5. USER MODEL (MODELS/USER.JS)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
 name: { type: String, required: true },
 email: { type: String, required: true, unique: true },
 password: { type: String, required: true },
 role: { type: String, enum: ['admin', 'user'], default: 'user' }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

6.PASSWORD HASHING AND REGISTRATION (CONTROLLERS/AUTHCONTROLLER.JS)

```
const bcrypt = require('bcryptjs');
const User = require('../models/User');

exports.register = async (req, res) => {
 try {
  const { name, email, password, role } = req.body;
  const salt = await bcrypt.genSalt(10);
  const hashedPassword = await bcrypt.hash(password, salt);

  const user = await User.create({ name, email, password: hashedPassword, role });
  res.status(201).json({ success: true, user: { id: user._id, email: user.email, role: user.role } });
 } catch (err) {
  res.status(400).json({ success: false, message: err.message });
 }
};
```

7. LOGIN AND JWT GENERATION (CONTROLLERS/AUTHCONTROLLER.JS)

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const User = require('../models/User');

exports.login = async (req, res) => {
 try {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.password))) {
   return res.status(401).json({ success: false, message: 'Invalid credentials' });
  }

  const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1d' });
  res.json({ success: true, token, role: user.role });
 } catch (err) {
  res.status(500).json({ success: false, message: err.message });
 }
};
```

8. AUTHENTICATION MIDDLEWARE (MIDDLEWARE/AUTH.JS)

```
const jwt = require('jsonwebtoken');

const auth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Access denied: No token' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Invalid token' });
  }
};

module.exports = auth;
```

9. CREATE API (CONTROLLERS/VESSELCONTROLLER.JS)

```
exports.createVessel = async (req, res) => {
  try {
    const vessel = await Vessel.create(req.body);
    res.status(201).json({ success: true, data: vessel });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
};
```

10. READ API (CONTROLLERS/VESSELCONTROLLER.JS)

```
exports.getVessels = async (req, res) => {
  try {
    const vessels = await Vessel.find();
    res.json({ success: true, count: vessels.length, data: vessels });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
};
```

11. UPDATE API (CONTROLLERS/VESSELCONTROLLER.JS)

```
exports.updateVessel = async (req, res) => {
  try {
    const vessel = await Vessel.findByIdAndUpdate(req.params.id, req.body, { new: true });
    if (!vessel) return res.status(404).json({ message: 'Vessel not found' });
    res.json({ success: true, data: vessel });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
};
```

12. DELETE API (CONTROLLERS/VESSELCONTROLLER.JS)

```
exports.deleteVessel = async (req, res) => {
  try {
    const vessel = await Vessel.findByIdAndDelete(req.params.id);
    if (!vessel) return res.status(404).json({ message: 'Vessel not found' });
    res.json({ success: true, message: 'Vessel deleted successfully' });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
};
```

13. EXPRESS ROUTES (ROUTES/API.JS)

```
const express = require('express');
const router = express.Router();
const auth = require('../middleware/auth');
const { register, login } = require('../controllers/authController');
const { getVessels, createVessel, updateVessel, deleteVessel } = require('../controllers/vesselController');

router.post('/auth/register', register);
router.post('/auth/login', login);

router.get('/vessels', auth, getVessels);
router.post('/vessels', auth, createVessel);
router.put('/vessels/:id', auth, updateVessel);
router.delete('/vessels/:id', auth, deleteVessel);

module.exports = router;
```

14. MAIN SERVER FILE (SERVER.JS)

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
```

```
const connectDB = require('./config/db');
const routes = require('./routes/api');

const app = express();
connectDB();

app.use(cors());
app.use(express.json());
app.use('/api', routes);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

15. REACT LOGIN COMPONENT (SRC/COMPONENTS/LOGIN.JSX)

```
import React, { useState } from 'react';

export default function Login({ onLoginSuccess }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });
    const data = await res.json();
    if (data.token) {
      localStorage.setItem('token', data.token);
      onLoginSuccess(data.role);
    } else {
      alert(data.message || 'Login failed');
    }
  };

  return (
    <form onSubmit={handleSubmit} className="p-4 max-w-sm mx-auto space-y-3">
      <input type="email" placeholder="Email" value={email} onChange={e => setEmail(e.target.value)} required
className="w-full p-2 border rounded" />
      <input type="password" placeholder="Password" value={password} onChange={e =>
setPassword(e.target.value)} required className="w-full p-2 border rounded" />
      <button type="submit" className="w-full bg-blue-600 text-white p-2 rounded">Sign In</button>
    </form>
  );
}
```

16. ADMIN DASHBOARD LOGIC (SRC/PAGES/DASHBOARD.JSX)

```
import React, { useState, useEffect } from 'react';

export default function Dashboard() {
  const [vessels, setVessels] = useState([]);

  useEffect(() => {
    fetch('/api/vessels', {
      headers: { Authorization: `Bearer ${localStorage.getItem('token')}` }
    })
      .then(res => res.json())
      .then(data => setVessels(data.data || []));
  }, []);

  return (
    <div className="p-6">
      <h1 className="text-xl font-bold mb-4">Fleet Operations ({vessels.length})</h1>
      <ul>
        {vessels.map(v => (
          <li key={v._id} className="p-2 border-b">{v.name} - {v.status}</li>
        ))}
      </ul>
    </div>
  );
}
```

17. REST API REQUEST AND RESPONSE EXAMPLES

Request:

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{
  "email": "admin@shipfleet.com",
  "password": "Admin@123"
}
```

Response:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "admin"
}
```

SCREENSHOTS OF THE PROJECT



Figure 5.1: Public Maritime Landing Page and Fleet Operations Gateway

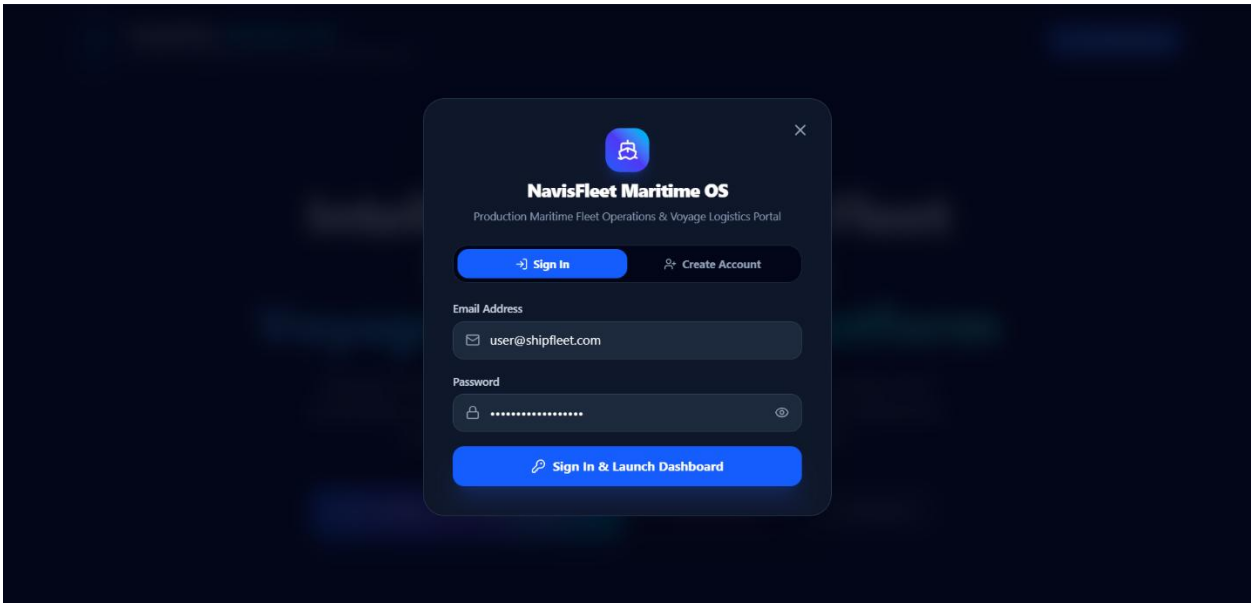


Figure 5.2: Role-Based Authentication & Modal Gate

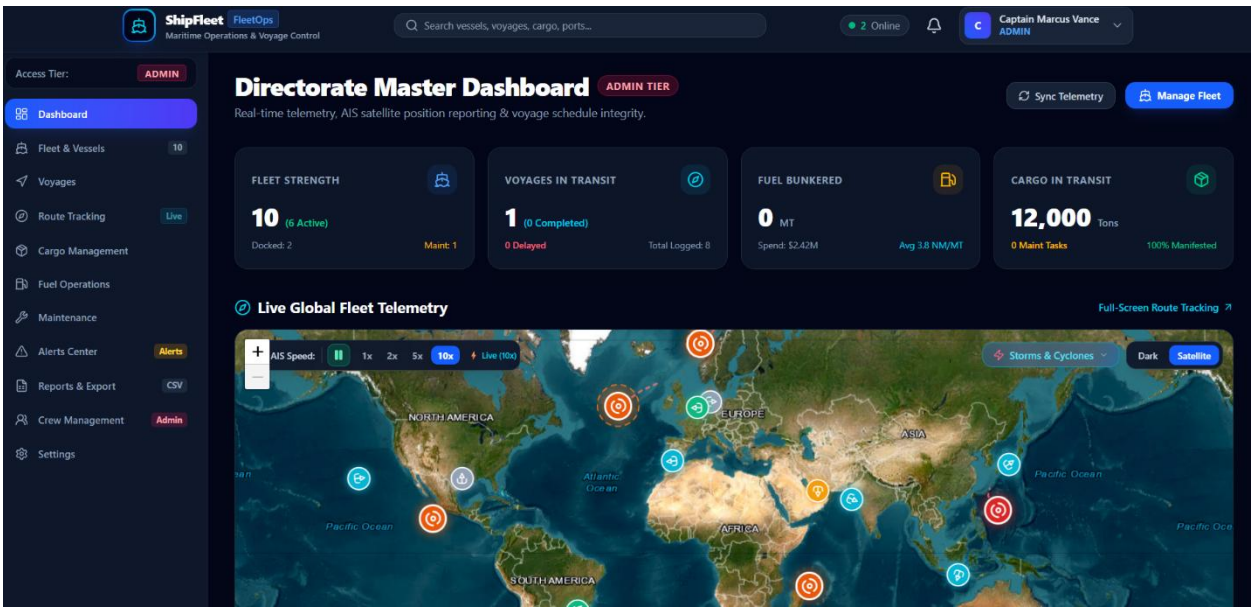


Figure 5.3: Directorate Master Dashboard with Real-Time KPI Cards & Telemetry

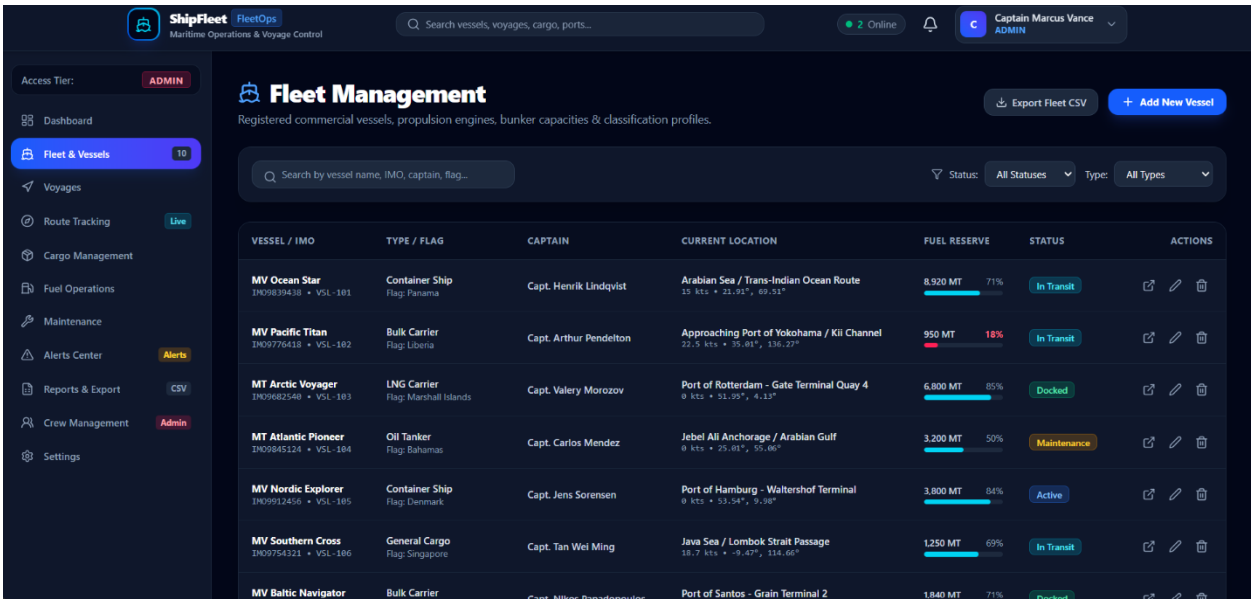


Figure 5.4: Fleet Strength & Vessel Inventory Management Console

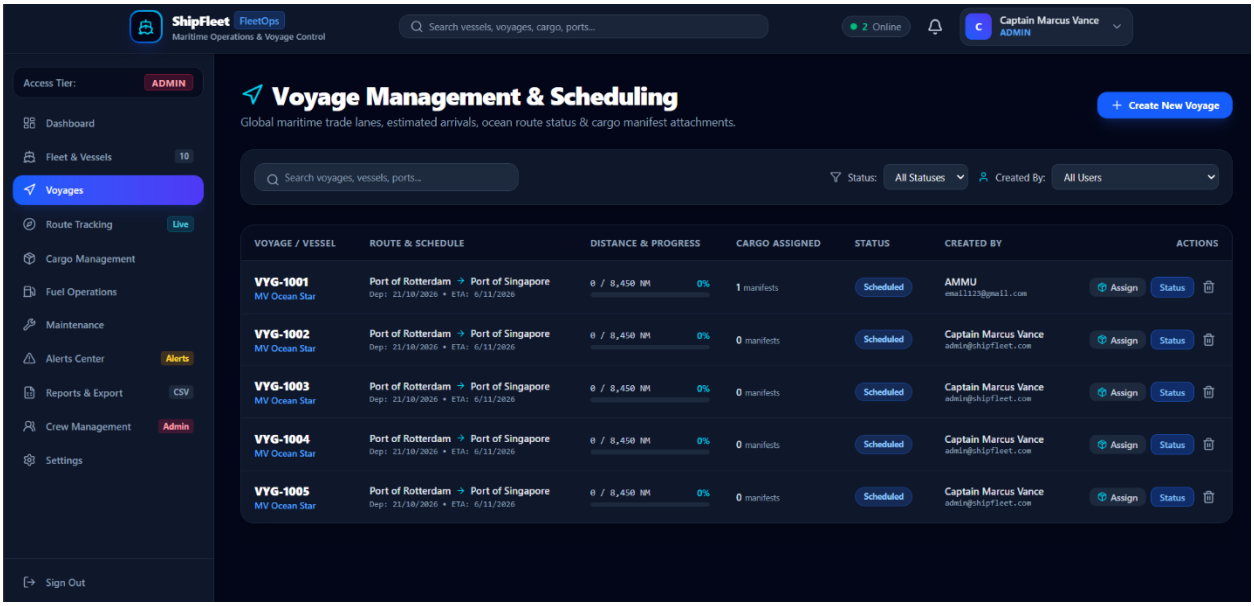


Figure 5.5: International Voyage Scheduling and Cargo Dispatch Interface



Figure 5.6: Interactive Maritime GIS Map with Live Weather & Telemetry Overlays

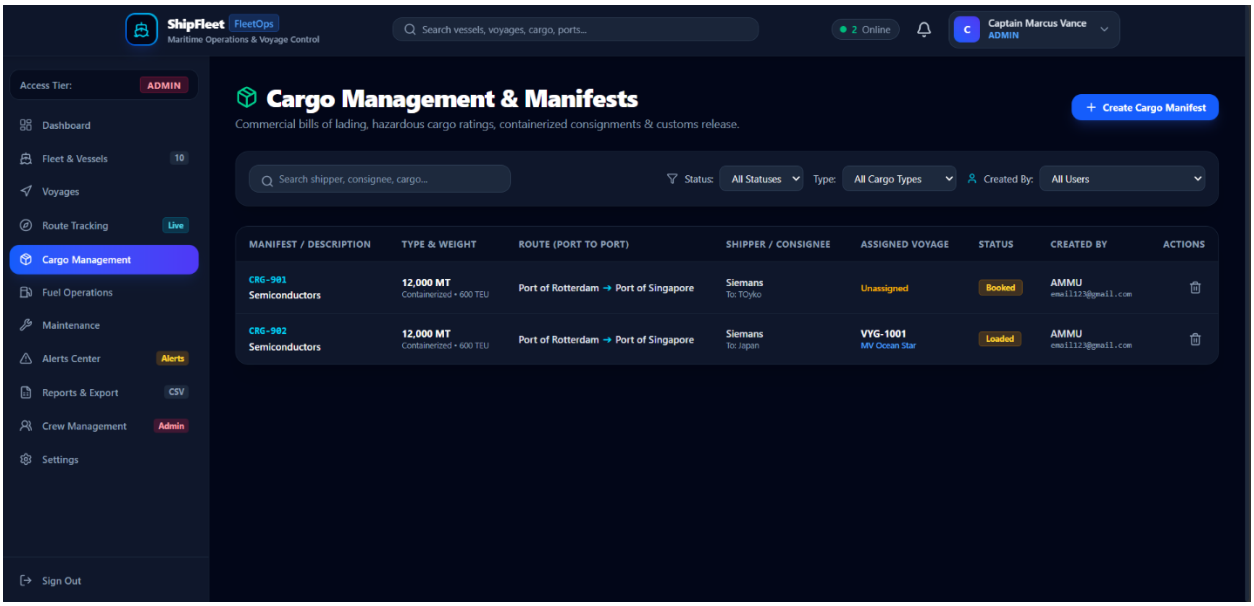


Figure 5.7: Cargo Manifest Management & Bill of Lading Roster

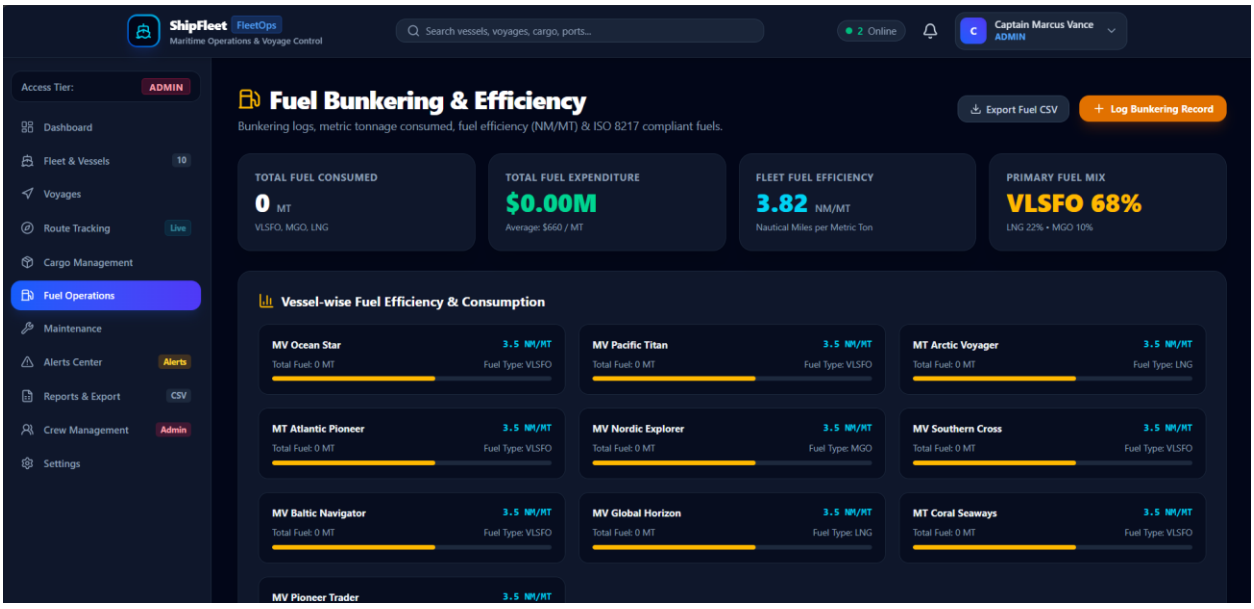


Figure 5.8: Bunker Fuel Consumption, Efficiency Metrics & Cost Analytics

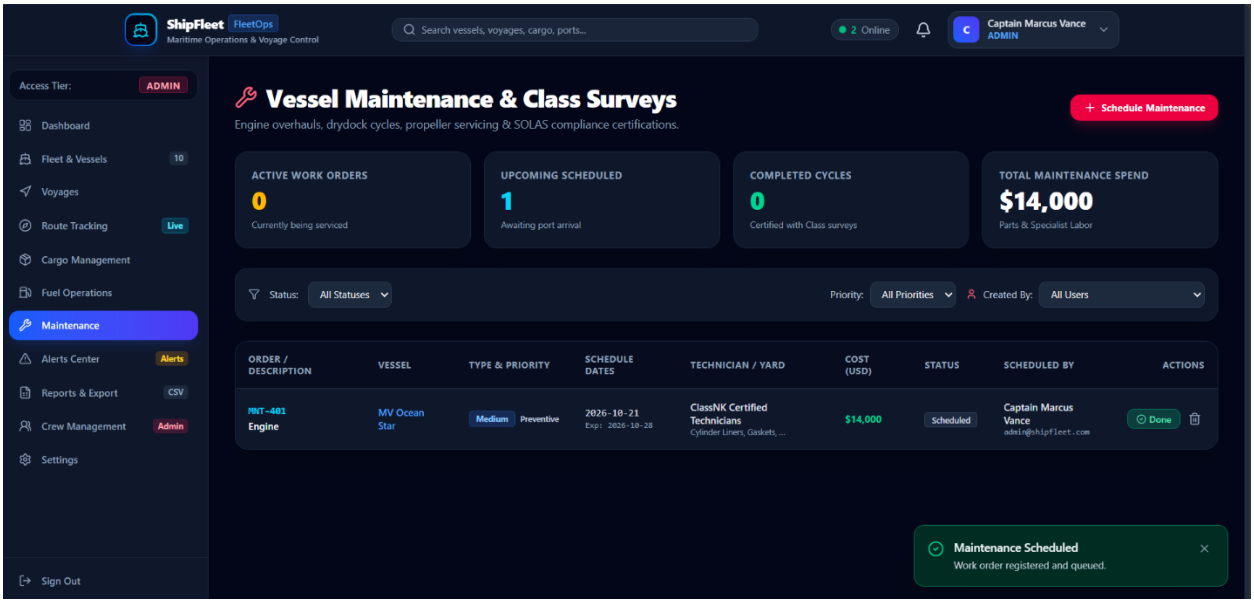


Figure 5.9: Preventative & Corrective Fleet Maintenance Work Orders

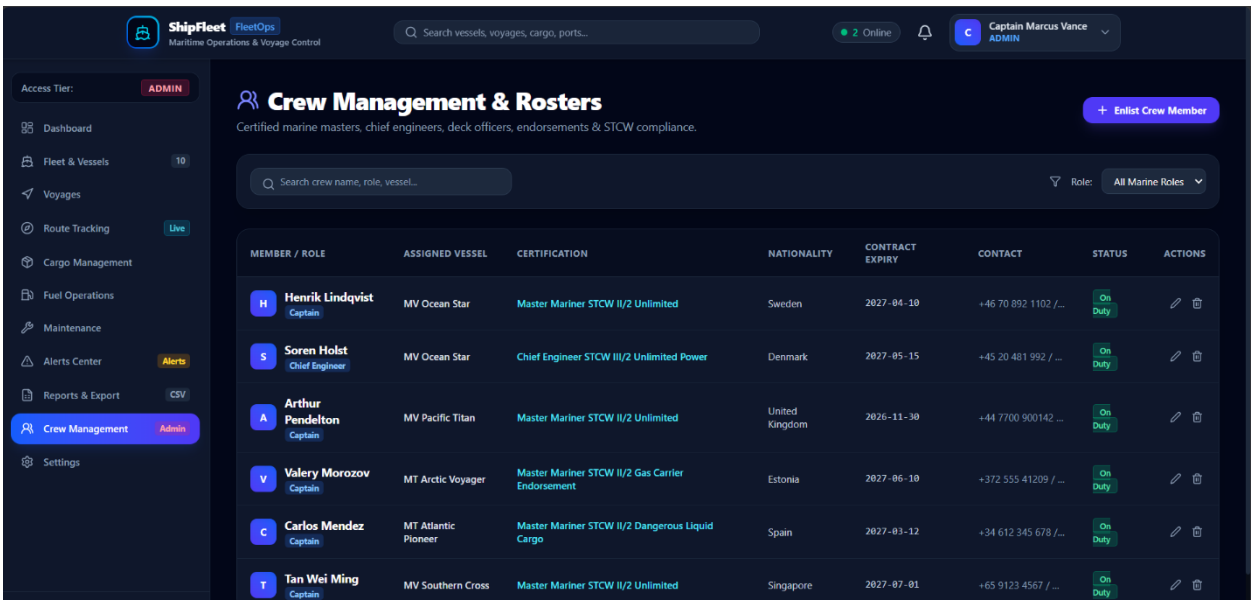


Figure 5.10: Marine Crew Roster & STCW Certification Compliance Directory

REFERENCES

1. International Maritime Organization (IMO), "Guidelines on Operational Measures for the Reduction of GHG Emissions from Ships," Resolution MEPC.336(76), 2021.
2. UNCTAD, "Review of Maritime Transport 2023: Navigating stormy waters," United Nations Publications, Geneva, 2023.
3. B. J. Tetreault, "Use of the Automatic Identification System (AIS) in Maritime Vessel Traffic Services," US Coast Guard Navigation Center, 2005.
4. Leaflet Documentation, https://leafletjs.com/, 2024.
5. Esri ArcGIS Online, "World Dark Gray Canvas Basemaps," https://www.arcgis.com/, 2024.
6. RainViewer Weather Maps API, https://www.rainviewer.com/api.html, 2024.
7. R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral Dissertation, University of California, Irvine, 2000.
8. Socket.io Documentation, https://socket.io/docs/v4/, 2024.